



Internship Project Report

- Yash Abhijeet Karve



24MEB0A69, B.TECH. MECHANICAL ENGINEERING, NIT - WARANGAL
PERFORMED AT EKA MOBILITY, CHAKAN, PUNE
FROM 5TH MAY 2026 TO 3RD JULY 2026

CAE: Scope and Overview

About Eka:

Company Overview:

EKA Mobility is a premier Indian Original Equipment Manufacturer (OEM) of electric commercial vehicles (EVCVs) headquartered in Pune, Maharashtra. The company is dedicated to accelerating the adoption of sustainable mobility by designing, developing, and manufacturing a comprehensive range of electric commercial vehicles for passenger and cargo transportation. With a strong emphasis on innovation, safety, and energy efficiency, EKA Mobility aims to provide reliable zero-emission transportation solutions that meet the evolving needs of urban and intercity mobility.

The company's product portfolio has India's largest range of EVCVs, including small commercial vehicles (SCVs) such as three-seater and six-seater passenger vehicles, three-wheeler cargo vehicles, and 1.5-tonne pickup trucks. Its bus vehicle platform lineup includes 7 m, 9 m, and 12 m city buses, a 12 m low-floor city bus, a 13.5 m coach bus available in both seater and sleeper configurations, and electric trucks in 55-tonne categories. This diverse product range enables the company to cater to public transport authorities, fleet operators, logistics providers, and commercial transportation businesses as well as individual owner-operators.

EKA Mobility places significant emphasis on research and development to enhance vehicle performance, safety, reliability, and passenger comfort. The company integrates modern engineering practices, advanced design methodologies, computer-aided engineering (CAE), and rigorous physical validation processes throughout the vehicle development cycle to ensure compliance with applicable automotive standards and regulations.

Driven by its vision of promoting clean and sustainable transportation, EKA Mobility continues to contribute to India's transition toward electric mobility through the development of technologically advanced commercial vehicles that combine environmental responsibility with operational efficiency.

Objectives of CAE:

Computer-Aided Engineering (CAE) is used to simulate, analyse, and optimize the performance of engineering designs before physical prototypes are built. Its primary objective is to improve product quality, reduce development time, and minimize manufacturing costs through virtual testing. In mechanical engineering, CAE helps evaluate structural strength, thermal behaviour, fluid flow, vibration, and durability under various operating conditions. It enables engineers to identify potential design issues early, leading to safer, more reliable, and efficient products.

Various Types of Analysis in Computer-Aided Engineering (CAE):

Computer-Aided Engineering (CAE) encompasses a wide range of simulation techniques that enable engineers to evaluate the performance, reliability, and safety of products under different operating conditions. Depending on the engineering problem, different types of analyses are performed to predict the behaviour of components and systems before manufacturing. Some of the most used CAE analyses are described below.

1. Durability Analysis:

Durability analysis evaluates the long-term performance of components under realistic operating conditions. It combines structural loads, fatigue calculations, road load data, and material properties to estimate product life. This analysis is extensively used in the automotive industry for suspension systems, chassis, engine mounts, and body structures to ensure long-term reliability.

a. Static Stiffness Analysis:

Static stiffness analysis evaluates the response of a structure under constant or slowly varying loads where inertial effects are negligible. It is commonly used to determine stiffness and deformations in machine components, brackets, frames, and pressure vessels.

b. Structural Strength Analysis:

Structural analysis is used to determine how a component or assembly behaves when subjected to external loads, pressures, forces, or constraints. It evaluates parameters such as stress, strain, displacement, factor of safety and reaction forces to ensure that the design can withstand operating conditions without failure. Structural analysis may be linear or nonlinear depending on the material properties and loading conditions.

c. Fatigue Analysis:

Fatigue analysis predicts the life of a component subjected to repeated or cyclic loading. Even when stresses are below the material's yield strength, repeated loading can initiate cracks that eventually lead to failure. Fatigue analysis estimates the number of load cycles a component can withstand and helps improve durability and service life.

d. Modal Analysis:

Modal analysis determines the natural frequencies and corresponding mode shapes of a structure. It helps engineers avoid resonance by ensuring that the operating frequency does not coincide with the natural frequency of the component. This analysis is important for rotating machinery, vehicle components, turbines, and aerospace structures.

e. Dynamic Analysis:

Dynamic analysis studies the behaviour of components subjected to time-dependent loads. It considers the effects of inertia and damping and includes analyses such as transient response, harmonic response, random vibration, and shock loading. Dynamic analysis is widely used in automotive, aerospace, and machinery applications.

2. NVH (Noise, Vibration, and Harshness) Analysis:

NVH analysis evaluates the vibration and acoustic performance of a product. It helps engineers identify sources of noise and vibration that may affect comfort, performance, or structural integrity. NVH simulations are particularly important in automotive, aerospace, and consumer product industries to improve user experience and reduce unwanted noise.

3. Multibody Dynamics (MBD):

Multi-body Dynamics analysis simulates the motion and interaction of interconnected rigid or flexible bodies. It evaluates displacement, velocity, acceleration, and forces acting on mechanical systems such as suspension mechanisms, robotic arms, gear trains, and linkages. MBD helps engineers optimize motion and improve mechanical performance.

4. Crash Analysis:

Crash analysis simulates the behaviour of vehicles or structures during collision events. It evaluates deformation, energy absorption, occupant safety, and structural integrity under high-speed impacts. Crash simulations help manufacturers design safer vehicles while reducing the number of expensive physical crash tests. Explicit finite element solvers are commonly used for crash analysis. Here at Eka, rollover testing in accordance with AIS-031 is conducted to assess the structural strength of bus superstructures and verify that adequate occupant survival space is maintained during rollover accidents. Also various homologation certifications test like

5. Computational Fluid Dynamics (CFD):

Computational Fluid Dynamics (CFD) is used to analyse the flow of liquids and gases around or through a component. It predicts parameters such as velocity, pressure, temperature,

turbulence, the coefficient of drag in aerodynamic analysis and flow patterns. CFD is widely used in the design of aircraft, automobiles, pumps, turbines, heat ex-changers, HVAC systems, and pipelines to improve aerodynamic and thermal performance.

a. Thermal Analysis:

Thermal analysis studies the distribution and transfer of heat within a component or system. It determines temperature variations, heat flux, and thermal stresses under steady-state or transient conditions. Thermal simulations are important in engine design, electronic cooling, batteries, furnaces, and heat ex-changers.

6. One-Dimensional (1D) Performance Analysis:

One-dimensional (1D) simulation models the behaviour of complete engineering systems using mathematical representations instead of detailed three-dimensional geometry. It is commonly used to simulate hydraulic, pneumatic, electrical, cooling, fuel, lubrication, and power-train systems. Here at Eka it is primarily used for range calculation of the vehicles. Since 1D simulation requires less computational effort than full 3D analysis, it is widely used during the early stages of product development for rapid design evaluation and system optimization

Approaches in CAE:

1. Validation:

Validation in Computer-Aided Engineering (CAE) is the process of evaluating the accuracy of simulation results by comparing them with experimental data, prototype testing, or established benchmarks. It ensures that the computational model reliably predicts the real-world behavior and performance of a component or vehicle under specified operating conditions.

2. Optimization Analysis:

Optimization analysis is used to improve the design by modifying dimensions, material properties, or geometry while satisfying engineering constraints. The objective may be to reduce weight, increase stiffness, improve durability, minimize cost, or enhance overall performance. Modern optimization techniques include topology optimization, shape optimization, and size optimization.

3. Explicit and Implicit Analysis:

Implicit analysis is generally used for static, low-speed dynamic, and nonlinear structural problems where the loading changes gradually. Explicit analysis is used for highly nonlinear and short-duration events such as crashes, impacts, explosions, metal forming, and drop tests, where large deformations and complex contact conditions are involved.

Applications of CAE Analysis

The different types of CAE analyses are extensively used across industries such as automotive, aerospace, railways, marine, heavy equipment, power generation, biomedical engineering, and manufacturing. These analyses enable engineers to validate designs, improve safety, optimize performance, reduce development costs, shorten product development cycles, and produce reliable products with minimal dependence on physical testing.

Pre-processing in CAE:

Pre-processing is the initial stage of the CAE workflow in which the analysis model is prepared. It involves creating or importing the geometry, assigning material properties, generating the finite element mesh, and applying boundary conditions and loads. Proper pre-processing ensures that the simulation accurately represents the real-world problem and provides reliable analysis results.

Post-Processing in CAE:

Post-processing is the final stage of the CAE process, where the simulation results are analyzed and interpreted. It involves evaluating quantities such as stress, strain, deformation, temperature, and displacement through graphical plots and reports. Post-processing helps engineers validate the design, identify critical regions, and make informed decisions for design improvement and optimization.

Python Automation for CAE Post-Processing

Post-processing in CAE

Postprocessing is the final stage of the CAE process, where the simulation results are analysed and interpreted. It involves evaluating quantities such as stress, strain, deformation, temperature, and displacement through graphical plots and reports. Postprocessing helps engineers validate the design, identify critical regions, and make informed decisions for design improvement and optimization.

Need for Automation in Post-processing:

Computer-Aided Engineering (CAE) simulations generate a significant amount of output data in the form of stresses, strains, displacements, reaction forces, temperatures, and other engineering parameters. As the complexity of products and the number of design iterations increase, the volume of simulation data also grows substantially. Managing and interpreting this data manually becomes increasingly difficult and inefficient.

The postprocessing stage involves extracting relevant information from the solver output files, interpreting the results, and preparing them for engineering evaluation. Performing these tasks manually for multiple simulation cases is repetitive, time-consuming, and prone to human errors. An automated workflow helps streamline these activities by processing large volumes of simulation data quickly and consistently.

Efficient organization of solver output data is equally important for effective engineering analysis. Automation enables simulation results to be systematically categorized, summarized, and stored in structured formats such as spreadsheets or databases. Well-organized data simplifies result comparison across different load cases and design iterations, allowing engineers to make informed design decisions more efficiently.

Automation also ensures consistency throughout the postprocessing process by applying the same extraction and reporting methodology to every simulation. This reduces the possibility of errors arising from manual data handling, improves the reliability of engineering reports, and significantly decreases the overall time required for result processing. Consequently, engineers can focus more on analysing results and optimizing designs rather than performing repetitive tasks.

Python plays a key role in automating CAE postprocessing because of its simplicity, flexibility, and extensive library support. In this project, Python was used to read the solver output files, extract the required engineering data, and automatically generate organized Microsoft Excel workbooks. The extracted data was further used to create graphs and visualizations, enabling quick interpretation and comparison of simulation results. By automating these tasks, Python reduced manual effort, improved accuracy, and significantly increased the efficiency of the overall postprocessing workflow.

HPC Data Processor – Program Documentation

Overview:

The Dashboard Process is a Python utility that converts log files into formatted Excel workbooks. It scans a specified folder for files named in YYYYMMDD format, extracts structured information from each record, and creates an Excel report for every input file.

Purpose:

- Read dashboard/log files from a user-specified folder.
- Parse each record into structured fields.
- Generate Excel reports with dynamic columns.
- Automatically format the Excel output for easier analysis.

Input:

- User enters the input folder path.
- Folder should contain files named in YYYYMMDD format (e.g., 20260629).

Processing Steps:

- Validate the input folder.
- Identify valid files.
- Create an output folder if needed.
- Read and parse each log file.
- Extract Date_Time, Event, Job_ID, and key=value pairs.
- Generate an Excel workbook.
- Apply formatting (bold headers, borders, auto column widths).
- Save the workbook in the output folder.

Output:

- An Excel (.xlsx) file is generated for each input file in the output folder.

Excel Worksheet:

- Worksheet name: PBS_Data
- Contains fixed columns (Date_Time, Event, Job_ID) and dynamically generated columns from key=value pairs.

Error Handling:

- Displays messages for invalid folders, missing files, read errors, and save errors.

Dependencies:

- Python 3.14.5
- Openpyxl

Summary:

Automates the conversion of dashboard log files into structured Excel reports for easy analysis.

Source Code:

The following is the source code for the HPC output file to excel file conversion, worked in Python 3.14.5 and Microsoft Visual Studio Code 64bit.

```
import os
import re
from openpyxl import Workbook
from openpyxl.styles import Border, Side, Font
from openpyxl.utils import get_column_letter

def convert_file_to_excel():

    # =====
    # GET INPUT FILE
    # =====

    input_folder = input(
        "Enter input folder path: "
    ).strip().replace("'", "")

    if not os.path.isdir(input_folder):

        print("\nERROR: Folder not found. ")
        print("Path entered: ", input_folder)
        return

    files = sorted([
        f for f in os.listdir(input_folder)
        if re.fullmatch(r"\d{8}", f)
    ])

    if not files:

        print("\nNo YYYYMMDD files found. ")
        return

    for filename in files:
        input_file = os.path.join(
            input_folder,
            filename
        )

        print("\nProcessing:", filename)

        # =====
        # CREATE OUTPUT FOLDER
        # =====

        parent_folder = os.path.dirname(input_file)

        output_folder = os.path.join(parent_folder, "output")

        os.makedirs(output_folder, exist_ok=True)

        # =====
        # OUTPUT EXCEL FILE NAME
        # =====

        base_name = os.path.basename(input_file)

        excel_file = os.path.join(
            output_folder,
            base_name + ".xlsx"
        )
```

```

# =====
# READ INPUT FILE
# =====

try:
    with open(
        input_file,
        "r",
        encoding="utf-8",
        errors="ignore"
    ) as file:

        content = file.read()

except Exception as e:
    print("Unable to read file.")
    print(e)
    return

# =====
# SPLIT RECORDS
# =====

records = [
    record.strip()
    for record in re.split(r"\r?\n", content)
    if record.strip()
]

all_rows = []
all_columns = []

```

```

# =====
# PARSE EACH RECORD
# =====

for record in records:

    parts = record.split(";", 3)

    row = {}

    if len(parts) > 0:
        row["Date_Time"] = parts[0]

    if len(parts) > 1:
        row["Event"] = parts[1]

    if len(parts) > 2:
        row["Job_ID"] = parts[2]

    if len(parts) > 3:

        remaining = parts[3].replace("\n", " ")

        matches = re.findall(
            r'([A-Za-z0-9_.]+)=(^[^s]+)',
            remaining
        )

        for key, value in matches:
            row[key] = value

    all_rows.append(row)

    for key in row:
        if key not in all_columns:
            all_columns.append(key)

# =====
# COLUMN ORDER
# =====

fixed_columns = [
    "Date_Time",
    "Event",
    "Job_ID"
]

dynamic_columns = [
    col for col in all_columns
    if col not in fixed_columns
]

columns = fixed_columns + dynamic_columns

# =====
# CREATE EXCEL
# =====

wb = Workbook()

ws = wb.active
ws.title = "PBS_Data"

```

```

# =====
# HEADER
# =====

for col_num, column_name in enumerate(columns, start=1):

    cell = ws.cell(
        row=1,
        column=col_num,
        value=column_name
    )

    cell.font = Font(bold=True)

# =====
# DATA
# =====

for row_num, row_data in enumerate(all_rows, start=2):

    for col_num, column_name in enumerate(columns, start=1):

        ws.cell(
            row=row_num,
            column=col_num,
            value=row_data.get(column_name, "")
        )

# =====
# ADD BORDERS
# =====

thin_border = Border(
    left=Side(style="thin"),
    right=Side(style="thin"),
    top=Side(style="thin"),
    bottom=Side(style="thin")
)

for row in ws.iter_rows():

    for cell in row:

        cell.border = thin_border

```

```

# =====
# AUTO-FIT COLUMNS
# =====

for column_cells in ws.columns:

    max_length = 0

    for cell in column_cells:

        try:
            if cell.value is not None:
                max_length = max(
                    max_length,
                    len(str(cell.value))
                )
        except:
            pass

    adjusted_width = min(max_length + 2, 60)

    ws.column_dimensions[
        get_column_letter(column_cells[0].column)
    ].width = adjusted_width

# =====
# SAVE FILE
# =====

try:

    wb.save(excel_file)

    print("\nSUCCESS")
    print("Excel file created:")
    print(excel_file)

except Exception as e:

    print("Error saving Excel file")
    print(e)

if __name__ == "__main__":
    convert_file_to_excel()

```

Results:

Following is the sample of the excel file generated of the raw data file logs that were received from the HPC unit.

Date_Time	Event	Job_ID	user	group	project	jobname	queue	ctime	qtime	etime	Resource_List.mem	Resource_List.mpiexecs	Resource_List.ncpus	Resource_List.nodeset	Resource_List.place	Resource_List.select	Resource_List.software
12/19/2025 10:15:07 +05:30	Q	1022.aui2025	dura1	dura	pbs_project_default	CFDanalysis	workq	1766119507	1766119507	1766119507	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 10:15:10 +05:30	S	1022.aui2025	dura1	dura	pbs_project_default	CFDanalysis	workq	1766119507	1766119507	1766119507	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 10:15:14 +05:30	E	1022.aui2025	dura1	dura	pbs_project_default	CFDanalysis	workq	1766119507	1766119507	1766119507	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 10:18:35 +05:30	Q	1023.aui2025	dura1	dura	pbs_project_default	Test_Furnace	workq	1766119715	1766119715	1766119715	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 10:18:37 +05:30	S	1023.aui2025	dura1	dura	pbs_project_default	Test_Furnace	workq	1766119715	1766119715	1766119715	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 10:18:48 +05:30	D	1023.aui2025															
12/19/2025 10:55:38 +05:30	E	1024.aui2025	dura1	dura	pbs_project_default	Test_Furnace	workq	1766119715	1766119715	1766119715	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 11:11:51 +05:30	Q	1024.aui2025	dfe1	dfe	pbs_project_default	test11912	dfe	1766122915	1766122915	1766122915	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 11:13:42 +05:30	S	1024.aui2025	dfe1	dfe	pbs_project_default	test11912	dfe	1766122915	1766122915	0	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 11:16:08 +05:30	D	1024.aui2025															
12/19/2025 11:18:51 +05:30	E	1024.aui2025	dfe1	dfe	pbs_project_default	test11912	dfe	1766122915	1766122915	0	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 11:28:11 +05:30	Q	1025.aui2025	dfe1	dfe	pbs_project_default	durantest	dfe	1766123771	1766123771	1766123771	480gb	48	48	1	ncxhost	1 ncpus=48 mem=480gb hcratch=190gb	Optistruct
12/19/2025 11:29:11 +05:30	S	1025.aui2025	dfe1	dfe	pbs_project_default	durantest	dfe	1766123771	1766123771	1766123771	480gb	48	48	1	ncxhost	1 ncpus=48 mem=480gb hcratch=190gb	Optistruct
12/19/2025 11:34:40 +05:30	Q	1026.aui2025	dfe1	dfe	pbs_project_default	HyperView	workq	1766124280	1766124280	1766124280	256mb	1	1	1	free	1 archlinux.ncpus=1 mem=256mb ndelstopp=1 ncpus=1	HyperView
12/19/2025 11:34:42 +05:30	S	1026.aui2025	dfe1	dfe	pbs_project_default	HyperView	workq	1766124280	1766124280	1766124280	256mb	1	1	1	free	1 archlinux.ncpus=1 mem=256mb ndelstopp=1 ncpus=1	HyperView
12/19/2025 11:37:25 +05:30	E	1025.aui2025	dfe1	dfe	pbs_project_default	durantest	dfe	1766123771	1766123771	1766123771	480gb	48	48	1	ncxhost	1 ncpus=48 mem=480gb hcratch=190gb	Optistruct
12/19/2025 11:37:38 +05:30	E	1026.aui2025	dfe1	dfe	pbs_project_default	HyperView	workq	1766124280	1766124280	1766124280	256mb	1	1	1	free	1 archlinux.ncpus=1 mem=256mb ndelstopp=1 ncpus=1	HyperView
12/19/2025 11:38:38 +05:30	Q	1027.aui2025	dfe1	dfe	pbs_project_default	radtest11	dfe	1766124510	1766124510	1766124510	480gb	48	48	1	ncxhost	1 ncpus=48 mem=480gb hcratch=190gb	Radoss
12/19/2025 11:38:34 +05:30	S	1027.aui2025	dfe1	dfe	pbs_project_default	radtest11	dfe	1766124510	1766124510	1766124510	480gb	48	48	1	ncxhost	1 ncpus=48 mem=480gb hcratch=190gb	Radoss
12/19/2025 11:40:31 +05:30	Q	1028.aui2025	dfe1	dfe	pbs_project_default	HyperView	workq	1766124631	1766124631	1766124631	256mb	1	1	1	free	1 archlinux.ncpus=1 mem=256mb ndelstopp=1 ncpus=1	HyperView
12/19/2025 11:40:31 +05:30	S	1028.aui2025	dfe1	dfe	pbs_project_default	HyperView	workq	1766124631	1766124631	1766124631	256mb	1	1	1	free	1 archlinux.ncpus=1 mem=256mb ndelstopp=1 ncpus=1	HyperView
12/19/2025 11:40:39 +05:30	E	1028.aui2025	dfe1	dfe	pbs_project_default	HyperView	workq	1766124631	1766124631	1766124631	256mb	1	1	1	free	1 archlinux.ncpus=1 mem=256mb ndelstopp=1 ncpus=1	HyperView
12/19/2025 11:40:40 +05:30	Q	1029.aui2025	dfe1	dfe	pbs_project_default	HyperView	workq	1766124659	1766124659	1766124659	256mb	1	1	1	free	1 archlinux.ncpus=1 mem=256mb ndelstopp=1 ncpus=1	HyperView
12/19/2025 11:41:00 +05:30	E	1029.aui2025	dfe1	dfe	pbs_project_default	HyperView	workq	1766124659	1766124659	1766124659	256mb	1	1	1	free	1 archlinux.ncpus=1 mem=256mb ndelstopp=1 ncpus=1	HyperView
12/19/2025 11:04:27 +05:30	Q	1030.aui2025	dfe1	dfe	pbs_project_default	RFUSO	dfe	1766126067	1766126067	1766126067	15gb	96	96	2	ncxhost	2 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=20gb	Acusolve
12/19/2025 12:05:48 +05:30	E	1029.aui2025	dfe1	dfe	pbs_project_default	HyperView	workq	1766124659	1766124659	1766124659	256mb	1	1	1	free	1 archlinux.ncpus=1 mem=256mb ndelstopp=1 ncpus=1	HyperView
12/19/2025 12:05:48 +05:30	E	1030.aui2025	dfe1	dfe	pbs_project_default	RFUSO	dfe	1766126067	1766126067	1766126067	15gb	96	96	2	ncxhost	2 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=20gb	Acusolve
12/19/2025 13:10:20 +05:30	E	1030.aui2025	dfe1	dfe	pbs_project_default	RFUSO	dfe	1766126067	1766126067	1766126067	15gb	96	96	2	ncxhost	2 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=20gb	Acusolve
12/19/2025 14:38:14 +05:30	Q	1031.aui2025	dfe1	dfe	pbs_project_default	RFUSO-SO	dfe	1766131294	1766131294	1766131294	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 14:38:34 +05:30	D	1027.aui2025															
12/19/2025 14:39:31 +05:30	E	1027.aui2025	dfe1	dfe	pbs_project_default	radtest11	dfe	1766124510	1766124510	1766124510	480gb	48	48	1	ncxhost	1 ncpus=48 mem=480gb hcratch=190gb	Radoss
12/19/2025 14:39:31 +05:30	S	1031.aui2025	dfe1	dfe	pbs_project_default	RFUSO-SO	dfe	1766131294	1766131294	1766131294	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 14:41:51 +05:30	Q	1032.aui2025	dfe1	dfe	pbs_project_default	RFUSO-SO	dfe	1766135515	1766135515	1766135515	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 14:50:34 +05:30	Q	1031.aui2025	dfe1	dfe	pbs_project_default	PumpSim10	dfe	1766136174	1766136174	1766136174	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 15:18:58 +05:30	E	1031.aui2025	dfe1	dfe	pbs_project_default	PumpSim10	dfe	1766136174	1766136174	1766136174	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 15:19:01 +05:30	S	1032.aui2025	dfe1	dfe	pbs_project_default	RFUSO-SO	dfe	1766135515	1766135515	1766135515	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 15:19:01 +05:30	E	1032.aui2025	dfe1	dfe	pbs_project_default	RFUSO-SO	dfe	1766135515	1766135515	1766135515	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 16:03:20 +05:30	E	1033.aui2025	dfe1	dfe	pbs_project_default	PumpSim10	dfe	1766136574	1766136574	1766136574	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 16:03:24 +05:30	E	1033.aui2025	dfe1	dfe	pbs_project_default	PumpSim10	dfe	1766136574	1766136574	1766136574	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 16:49:18 +05:30	Q	1034.aui2025	dfe1	dfe	pbs_project_default	PumpSim10-1	dfe	1766143158	1766143158	1766143158	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 16:49:21 +05:30	S	1034.aui2025	dfe1	dfe	pbs_project_default	PumpSim10-1	dfe	1766143158	1766143158	1766143158	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 16:58:08 +05:30	E	1035.aui2025	dfe1	dfe	pbs_project_default	PumpSim10rci	dfe	1766143686	1766143686	1766143686	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 19:03:18 +05:30	E	1034.aui2025	dfe1	dfe	pbs_project_default	PumpSim10-1	dfe	1766143158	1766143158	1766143158	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 19:03:21 +05:30	S	1035.aui2025	dfe1	dfe	pbs_project_default	PumpSim10rci	dfe	1766143686	1766143686	1766143686	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve
12/19/2025 19:45:11 +05:30	E	1035.aui2025	dfe1	dfe	pbs_project_default	PumpSim10rci	dfe	1766143686	1766143686	1766143686	1gb	144	144	0	ncxhost	0 ncpus=48 mpiexecs=48 ompthreads=1 mem=1gb hcratch=1gb	Acusolve

HPC Utilization Tracking Sheet – Program Documentation

Overview:

This Python utility consolidates daily HPC job execution reports into a weekly utilization report. It calculates cluster utilization, summarizes application-wise resource consumption, and generates an Excel workbook with formatted worksheets and charts.

Purpose:

- Process daily HPC Excel reports.
- Calculate weekly HPC core-hour utilization.
- Summarize application-wise resource usage.
- Generate charts for utilization trends and application usage.

Input:

- User provides the folder containing daily Excel (.xlsx) files.
- Input files must be named in YYYYMMDD format (e.g., 20260629.xlsx).
- Each workbook should contain a PBS_Data worksheet (or the first worksheet) with job execution details.

Processing Steps:

- Validate the input folder.
- Identify valid daily Excel files.
- Group files into weekly batches.
- Read required columns (Event, CPU count, Wall time, Software).
- Calculate core hours for completed jobs (Event = E).
- Aggregate weekly utilization and application-wise usage.
- Generate summary worksheets.
- Create bar and line charts for weekly and application utilization.
- Save the consolidated report.

Output:

- Creates HPC_Weekly_Utilization.xlsx in the input folder.
- Workbook contains Weekly Utilization and Application Utilization worksheets with charts.

Charts Generated:

- Weekly Usage vs Weekly Capacity (Hours).
- Weekly Utilization Percentage.
- Application-wise Hours Consumed.
- Application-wise Utilization Percentage.

Error Handling:

- Checks for invalid folder path.
- Skipping files that cannot be opened.
- Reports missing required columns.
- Continues processing remaining files when errors occur.

Dependencies:

1. Python 3.14.5
2. Openpyxl

Summary:

Automates HPC utilization reporting by consolidating daily execution data into a weekly dashboard with utilization metrics, application summaries, and graphical analysis.

Source Code:

```
1 import os
2 from datetime import datetime, time
3
4 from openpyxl import Workbook, load_workbook
5 from openpyxl.styles import Font, Border, side
6 from openpyxl.utils import get_column_letter
7 from openpyxl.chart import (
8     BarChart,
9     LineChart,
10    Reference
11 )
12
13 def find_column(headers, possible_names):
14     """
15     Finds a column by checking multiple possible names.
16     Returns the matching header name or None.
17     """
18
19     normalized = {
20         str(h).strip().lower(): h
21         for h in headers
22         if h is not None
23     }
24
25     for name in possible_names:
26         if name.lower() in normalized:
27             return normalized[name.lower()]
28
29     return None
30
31
32 def walltime_to_hours(value):
33     """
34     Converts walltime to decimal hours.
35     Supports:
36         HH:MM:SS
37         H:MM:SS
38     Excel time objects
39     """
40
41     if value is None:
42         return 0.0
43
44     if isinstance(value, time):
45         return (
46             value.hour
47             + value.minute / 60
48             + value.second / 3600
49         )
50
51     value = str(value).strip()
52
53     if not value:
54         return 0.0
55
56     try:
57         parts = value.split(":")
58
59         if len(parts) == 3:
60             h, m, s = map(int, parts)
61
62             return (
63                 h
64                 + m / 60
65                 + s / 3600
66             )
67
68     except:
69         pass
70
71     return 0.0
72
73
```



```

74 def create_hpc_utilization_report():
75
76     # =====
77     # USER INPUT
78     # =====
79
80     input_folder = input(
81         "Enter folder containing weekly Excel files: "
82     ).strip().replace("'", '')
83
84     if not os.path.isdir(input_folder):
85
86         print("\nERROR: Folder not found")
87         return
88
89     # =====
90     # FIND INPUT FILES
91     # =====
92
93     files = sorted([
94         f for f in os.listdir(input_folder)
95         if (
96             f.endswith(".xlsx")
97             and not f.startswith("~$")
98             and len(os.path.splitext(f)[0]) == 8
99             and os.path.splitext(f)[0].isdigit()
100         )
101     ])
102
103     if not files:
104
105         print("\nNo YYYYMMDD Excel files found.")
106         return
107
108     # =====
109     # CONSTANTS
110     # =====
111
112     WEEKLY_CAPACITY = 144 * 24 * 7
113
114     summary_rows = []
115
116     application_totals = {}
117
118     # =====
119     # PROCESS FILES
120     # =====
121
122     for i in range(0, len(files), 7):
123
124         weekly_files = files[i:i + 7]
125
126         weekly_usage = 0.0
127
128         for filename in weekly_files:
129
130             filepath = os.path.join(
131                 input_folder,
132                 filename
133             )
134
135             print(f"Processing {filename}")
136
137             try:
138
139                 wb = load_workbook(
140                     filepath,
141                     data_only=True
142                 )
143
144             except Exception as e:
145
146                 print(f"Unable to open {filename}")
147                 print(e)
148                 continue
149
150             # =====
151             # FIND PBS_DATA SHEET
152             # =====
153

```

```

154     if "PBS_Data" in wb.sheetnames:
155         |
156         |     ws = wb["PBS_Data"]
157         |
158         |     else:
159         |         |
160         |         |     ws = wb[wb.sheetnames[0]]
161         |         |
162         |         |     # =====
163         |         |     # READ HEADERS
164         |         |     # =====
165         |         |
166         |         |     headers = []
167         |         |
168         |         |     for cell in ws[1]:
169         |         |         |     headers.append(cell.value)
170         |         |
171         |         |     # =====
172         |         |     # FLEXIBLE COLUMN DETECTION
173         |         |     # =====
174         |         |
175         |         |     event_col = find_column(
176         |         |         |     headers,
177         |         |         |     [
178         |         |         |         "Event",
179         |         |         |         "event"
180         |         |         |     ]
181         |         |         )
182         |         |
183         |         |     ncpus_col = find_column(
184         |         |         |     headers,
185         |         |         |     [
186         |         |         |         "resources_used.ncpus",
187         |         |         |         "resources_used_ncpus",
188         |         |         |         "ncpus",
189         |         |         |         "Resource_List.ncpus"
190         |         |         |     ]
191         |         |         )
192         |         |
193         |         |     walltime_col = find_column(
194         |         |         |     headers,
195         |         |         |     [
196         |         |         |         "resources_used.walltime",
197         |         |         |         "resources_used_walltime",
198         |         |         |         "walltime"
199         |         |         |     ]
200         |         |         )
201         |         |
202         |         |     software_col = find_column(
203         |         |         |     headers,
204         |         |         |     [
205         |         |         |         "Resource_List.software",
206         |         |         |         "resource_list.software",
207         |         |         |         "software"
208         |         |         |     ]
209         |         |         )
210         |         |
211         |         |     if (
212         |         |         |     event_col is None
213         |         |         |     or ncpus_col is None
214         |         |         |     or walltime_col is None
215         |         |         |     or software_col is None
216         |         |         ):
217         |         |         |
218         |         |         |     print(
219         |         |         |         |     f"Required columns not found in {filename}"
220         |         |         |         )
221         |         |         |     continue
222         |         |
223         |         |     # =====
224         |         |     # HEADER INDEX LOOKUP
225         |         |     # =====
226         |         |

```

```

227 header_index = {
228     str(cell.value): idx
229     for idx, cell in enumerate(ws[1], start=1)
230     if cell.value is not None
231 }
232
233 event_idx = header_index[event_col]
234 ncpus_idx = header_index[ncpus_col]
235 walltime_idx = header_index[walltime_col]
236 software_idx = header_index[software_col]
237
238 # =====
239 # CALCULATE WEEKLY USAGE
240 # =====
241
242 for row in ws.iter_rows(
243     min_row=2,
244     values_only=True
245 ):
246
247     event = row[event_idx - 1]
248
249     if str(event).strip() != "E":
250         continue
251
252     try:
253
254         ncpus = float(
255             row[ncpus_idx - 1]
256         )
257
258     except:
259
260         ncpus = 0
261
262     walltime_hours = walltime_to_hours(
263         row[walltime_idx - 1]
264     )
265
266     core_hours = (
267         ncpus * walltime_hours
268     )
269
270     weekly_usage += core_hours
271
272     software = str(
273         row[software_idx - 1]
274     ).strip()
275
276     if not software:
277         software = "Unknown"
278
279     application_totals[software] = (
280         application_totals.get(
281             software,
282             0
283         )
284         + core_hours
285     )
286

```

```

287 # =====
288 # DATE FROM LAST FILE OF THE WEEK
289 # =====
290
291 file_date = datetime.strptime(
292     os.path.splitext(weekly_files[-1])[0],
293     "%Y%m%d"
294 )
295
296 as_on_date = file_date.strftime(
297     "%d-%b"
298 )
299
300 usage_percent = (
301     weekly_usage
302     / WEEKLY_CAPACITY
303     * 100
304 )
305
306 summary_rows.append(
307     [
308         as_on_date,
309         round(weekly_usage, 2),
310         WEEKLY_CAPACITY,
311         round(usage_percent, 2),
312         100.0
313     ]
314 )
315
316 # =====
317 # CREATE OUTPUT WORKBOOK
318 # =====
319
320 output_wb = Workbook()
321
322 ws = output_wb.active
323
324 ws.title = "Weekly Utilization"
325
326 app_ws = output_wb.create_sheet(
327     "Application Utilization"
328 )
329
330 headers = [
331     "As on Date",
332     "Weekly Usage (Hrs)",
333     "Weekly Capacity (Hrs)",
334     "Weekly Usage (%)",
335     "Weekly Capacity (%)"
336 ]
337
338 # =====
339 # HEADER
340 # =====
341
342 thin_border = Border(
343     left=Side(style="thin"),
344     right=Side(style="thin"),
345     top=Side(style="thin"),
346     bottom=Side(style="thin")
347 )
348
349 for col_num, header in enumerate(
350     headers,
351     start=1
352 ):
353
354     cell = ws.cell(
355         row=1,
356         column=col_num,
357         value=header
358     )
359
360     cell.font = Font(bold=True)
361     cell.border = thin_border
362

```

```

363 # =====
364 # DATA
365 # =====
366
367 for row_num, row_data in enumerate(
368     summary_rows,
369     start=2
370 ):
371
372     for col_num, value in enumerate(
373         row_data,
374         start=1
375     ):
376
377         cell = ws.cell(
378             row=row_num,
379             column=col_num,
380             value=value
381         )
382
383         cell.border = thin_border
384
385 # =====
386 # AUTO WIDTH
387 # =====
388
389 for column_cells in ws.columns:
390
391     max_length = 0
392
393     for cell in column_cells:
394
395         try:
396
397             if cell.value is not None:
398
399                 max_length = max(
400                     max_length,
401                     len(str(cell.value))
402                 )
403
404         except:
405             pass
406
407     ws.column_dimensions[
408         get_column_letter(
409             column_cells[0].column
410         )
411     ].width = min(
412         max_length + 3,
413         30
414     )
415
416 ws.freeze_panes = "A2"
417

```

```

418 # =====
419 # APPLICATION UTILIZATION SHEET
420 # =====
421
422 app_headers = [
423     "Application",
424     "Hours Consumed",
425     "Usage (%)"
426 ]
427
428 for col_num, header in enumerate(
429     app_headers,
430     start=1
431 ):
432
433     cell = app_ws.cell(
434         row=1,
435         column=col_num,
436         value=header
437     )
438
439     cell.font = Font(bold=True)
440     cell.border = thin_border
441
442 total_hours = sum(
443     application_totals.values()
444 )
445
446 sorted_apps = sorted(
447     application_totals.items(),
448     key=lambda x: x[1],
449     reverse=True
450 )
451
452 app_row = 2
453
454 for app, hours in sorted_apps:
455
456     usage_pct = (
457         hours / total_hours * 100
458     ) if total_hours else 0
459
460     app_ws.cell(
461         app_row,
462         1,
463         app
464     )
465
466     app_ws.cell(
467         app_row,
468         2,
469         round(hours, 2)
470     )
471
472     app_ws.cell(
473         app_row,
474         3,
475         round(usage_pct, 2)
476     )
477
478     for col in range(1, 4):
479
480         app_ws.cell(
481             app_row,
482             col
483         ).border = thin_border
484
485     app_row += 1
486
487 app_ws.cell(
488     app_row,
489     1,
490     "Total"
491 )
492

```

```

493     app_ws.cell(
494         app_row,
495         2,
496         round(total_hours, 2)
497     )
498
499     app_ws.cell(
500         app_row,
501         3,
502         100
503     )
504
505     for col in range(1, 4):
506
507         app_ws.cell(
508             app_row,
509             col
510         ).border = thin_border
511
512     last_app_row = app_row
513
514     for column_cells in app_ws.columns:
515
516         max_length = 0
517
518         for cell in column_cells:
519
520             if cell.value is not None:
521
522                 max_length = max(
523                     max_length,
524                     len(str(cell.value))
525                 )
526
527         app_ws.column_dimensions[
528             get_column_letter(
529                 column_cells[0].column
530             )
531         ].width = min(
532             max_length + 3,
533             30
534         )
535

```

```

536 # =====
537 # CHART CATEGORY REFERENCE
538 # =====
539
540 cats = Reference(
541     ws,
542     min_col=1,
543     min_row=2,
544     max_row=len(summary_rows) + 1
545 )
546
547 # =====
548 # CHART 1 - HOURS
549 # =====
550
551 bar_chart = BarChart()
552
553 bar_chart.type = "col"
554 bar_chart.style = 10
555
556 bar_chart.title = (
557     "Eka HPC Utilization Tracker (Hours)"
558 )
559
560 bar_chart.y_axis.title = (
561     "Weekly Usage (Hrs)"
562 )
563
564 bar_chart.x_axis.title = (
565     "As on Date"
566 )
567
568 usage_data = Reference(
569     ws,
570     min_col=2,
571     max_col=2,
572     min_row=1,
573     max_row=len(summary_rows) + 1
574 )
575
576 bar_chart.add_data(
577     usage_data,
578     titles_from_data=True
579 )
580
581 bar_chart.set_categories(cats)
582
583 line_chart = LineChart()
584
585 capacity_data = Reference(
586     ws,
587     min_col=3,
588     max_col=3,
589     min_row=1,
590     max_row=len(summary_rows) + 1
591 )
592
593 line_chart.add_data(
594     capacity_data,
595     titles_from_data=True
596 )
597
598 line_chart.y_axis.axId = 200
599 line_chart.y_axis.crosses = "max"
600
601 bar_chart += line_chart
602
603 bar_chart.height = 12
604 bar_chart.width = 24
605
606 ws.add_chart(
607     bar_chart,
608     "G2"
609 )
610

```



```

611 # =====
612 # CHART 2 - UTILIZATION %
613 # =====
614
615 bar_chart2 = BarChart()
616
617 bar_chart2.type = "col"
618 bar_chart2.style = 10
619
620 bar_chart2.title = (
621     "Eka HPC Utilization Tracker (%)"
622 )
623
624 bar_chart2.y_axis.title = (
625     "Weekly Usage (%)"
626 )
627
628 bar_chart2.x_axis.title = (
629     "As on Date"
630 )
631
632 usage_pct = Reference(
633     ws,
634     min_col=4,
635     max_col=4,
636     min_row=1,
637     max_row=len(summary_rows) + 1
638 )
639
640 bar_chart2.add_data(
641     usage_pct,
642     titles_from_data=True
643 )
644
645 bar_chart2.set_categories(cats)
646
647 line_chart2 = LineChart()
648
649 capacity_pct = Reference(
650     ws,
651     min_col=5,
652     max_col=5,
653     min_row=1,
654     max_row=len(summary_rows) + 1
655 )
656
657 line_chart2.add_data(
658     capacity_pct,
659     titles_from_data=True
660 )
661
662 line_chart2.y_axis.axId = 300
663 line_chart2.y_axis.crosses = "max"
664
665 bar_chart2 += line_chart2
666
667 bar_chart2.height = 12
668 bar_chart2.width = 24
669
670 ws.add_chart(
671     bar_chart2,
672     "G24"
673 )
674

```

```

675 # =====
676 # APPLICATION HOURS CHART
677 # =====
678
679 app_chart1 = BarChart()
680
681 app_chart1.type = "col"
682 app_chart1.style = 10
683
684 app_chart1.title = (
685     "Application-wise Usage (Hours)"
686 )
687
688 app_chart1.y_axis.title = (
689     "Hours Consumed"
690 )
691
692 app_chart1.x_axis.title = (
693     "Application"
694 )
695
696 data = Reference(
697     app_ws,
698     min_col=2,
699     max_col=2,
700     min_row=1,
701     max_row=last_app_row - 1
702 )
703
704 cats = Reference(
705     app_ws,
706     min_col=1,
707     min_row=2,
708     max_row=last_app_row - 1
709 )
710
711 app_chart1.add_data(
712     data,
713     titles_from_data=True
714 )
715
716 app_chart1.set_categories(cats)
717
718 app_chart1.height = 12
719 app_chart1.width = 20
720
721 app_ws.add_chart(
722     app_chart1,
723     "F2"
724 )
725

```

```

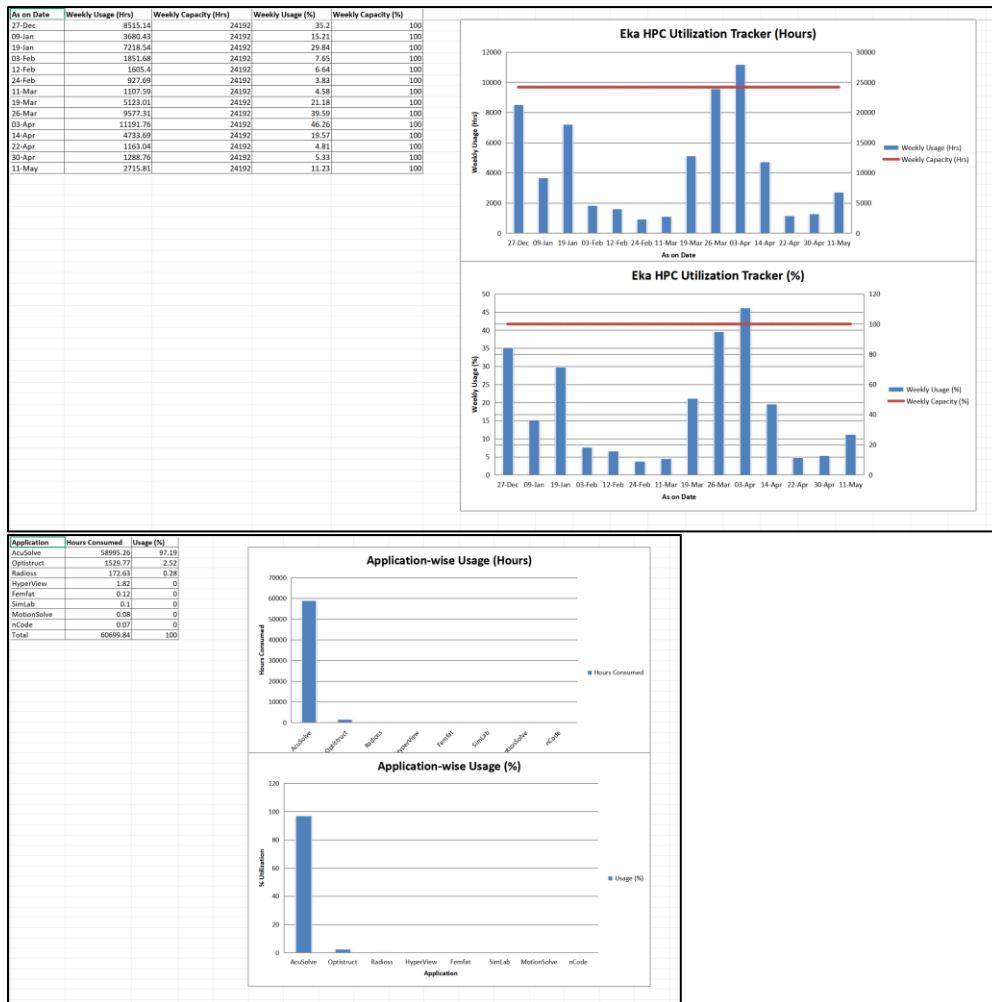
726 # =====
727 # APPLICATION % CHART
728 # =====
729
730 app_chart2 = BarChart()
731
732 app_chart2.type = "col"
733 app_chart2.style = 10
734
735 app_chart2.title = (
736     "Application-wise Usage (%)"
737 )
738
739 app_chart2.y_axis.title = (
740     "% Utilization"
741 )
742
743 app_chart2.x_axis.title = (
744     "Application"
745 )
746
747 data = Reference(
748     app_ws,
749     min_col=3,
750     max_col=3,
751     min_row=1,
752     max_row=last_app_row - 1
753 )
754
755 app_chart2.add_data(
756     data,
757     titles_from_data=True
758 )
759
760 app_chart2.set_categories(cats)
761
762 app_chart2.height = 12
763 app_chart2.width = 20
764
765 app_ws.add_chart(
766     app_chart2,
767     "F22"
768 )
769
770 # =====
771 # SAVE
772 # =====
773
774 output_file = os.path.join(
775     input_folder,
776     "HPC_Weekly_Utilization.xlsx"
777 )
778
779 output_wb.save(output_file)
780
781 print("\nSUCCESS")
782 print("Output file:")
783 print(output_file)
784
785
786 if __name__ == "__main__":
787     create_hpc_utilization_report()

```

Results:

The automation scripts developed for HPC tracking significantly reduced the time required for postprocessing and data organization. Previously, extracting data from each solver output file and manually entering it into Excel required approximately **2-3 hours per output file**. After processing all the output files, an additional **2-3 hours** were needed to consolidate the extracted information and prepare the final HPC tracking sheet.

By automating the entire workflow using Python, the process of extracting data, generating Excel workbooks, organizing the results, and creating the final tracking sheet was completed in **under 5 minutes** for all output files combined. The automation also eliminated manual data entry, reduced the possibility of human errors, ensured consistency across all processed files, and enabled engineers to obtain standardized reports almost instantly. This substantial reduction in processing time greatly improved productivity and allowed engineers to focus on result interpretation and design optimization rather than repetitive post-processing tasks.



Automation of OptiStruct Modal Analysis

Postprocessing Using Python:

Introduction:

Modal analysis is one of the most important analyses performed in Computer-Aided Engineering (CAE) to determine the dynamic characteristics of a structure. It is used to calculate the natural frequencies and corresponding mode shapes of a component, enabling engineers to evaluate its vibration behaviour and avoid resonance during operation. The results of modal analysis are generated by the OptiStruct solver in the form of output files containing detailed numerical information.

Although these output files provide all the necessary data, extracting the required information manually is time-consuming, particularly when a large number of simulations are performed. Engineers often need only a concise summary containing key modal characteristics such as the number of rigid body modes, the first flexible mode, and the corresponding natural frequency. Automating this process improves efficiency and reduces the effort involved in reviewing solver outputs.

Objective:

The objective of this project was to develop a Python-based automation tool capable of reading OptiStruct solver output (.out) files and extracting important modal analysis information. The tool automatically identifies the number of rigid body modes, determines the first flexible mode, extracts the corresponding natural frequency, and presents the results in a structured HTML summary report for quick review by engineers. The HTML summary report is formatted as read-only and also has HASH (sha256) verification id for prevention of data tampering

Methodology:

The developed Python script reads the OptiStruct solver output (.out) file and searches for the modal analysis section containing the eigenvalue results. Using pattern matching techniques, the script extracts the mode numbers and their corresponding natural frequencies from the output data.

The extracted information is then analysed to identify the rigid body modes, which ideally possess zero or near-zero natural frequencies. The script determines the first flexible mode by locating the first significant non-zero natural frequency after the rigid modes. Based on the extracted results, a concise summary is generated containing the total number of rigid modes, the mode number at which the first natural frequency occurs, and the value of the first natural frequency.

Finally, the extracted information is automatically formatted into an HTML report. The report presents the results in a clean and organized layout that can be viewed directly in any web browser, allowing engineers to quickly verify the quality of the modal analysis without manually searching through lengthy solver output files.

Role of Python:

Python was used to automate the complete post-processing workflow of the modal analysis results. It was employed to parse the OptiStruct output file, extract the required modal information using string processing and regular expressions, perform the necessary logical checks to identify rigid body modes, and generate an HTML summary report automatically.

The use of Python eliminated the need for manual inspection of large solver output files, improved the accuracy of data extraction, and significantly reduced the time required to review simulation results. The generated HTML report provides a user-friendly summary that can be easily shared and accessed through any standard web browser.

Dependencies:

1. Python 3.14.5
2. Openpyxl Library
3. Shutil Library
4. Hashlib Library
5. HTML 5

Source Code:

```
1 import os
2 import shutil
3 import hashlib
4 from datetime import datetime
5
6 def check_and_move_file():
7
8     # =====
9     # USER INPUT
10    # =====
11
12    folder = input("Enter main folder path: ")
13
14    pre_path = os.path.join(folder, "pre-process")
15    post_path = os.path.join(folder, "post-process")
16
17    # =====
18    # CREATE POST-PROCESS FOLDER IF NOT PRESENT
19    # =====
20
21    os.makedirs(post_path, exist_ok=True)
22
23    try:
24
25        # =====
26        # CHECK PRE-PROCESS FOLDER
27        # =====
28
29        if not os.path.exists(pre_path):
30
31            print(f"\nERROR: pre-process folder not found:\n{pre_path}")
32            return
33
34        # =====
35        # FIND .OUT FILES
36        # =====
37
38        files = [
39            f for f in os.listdir(pre_path)
40            if f.endswith(".out")
41        ]
42
43        if not files:
44
45            print("\nNo .out file found in pre-process folder.")
46            return
47
48        # =====
49        # SELECT FIRST OUT FILE
50        # =====
51
52        filename = files[0]
53
54        file_path = os.path.join(pre_path, filename)
55
56        # =====
57        # READ ORIGINAL OUT FILE
58        # =====
59
60        with open(file_path, 'r') as file:
61
62            lines = file.readlines()
63
```



```

64 # =====
65 # FLEXIBLE HEADER DETECTION
66 # =====
67
68 keywords = ["Subcase", "Mode", "Frequency"]
69
70 start_index = -1
71
72 for i, line in enumerate(lines):
73     if all(word in line for word in keywords):
74         start_index = i + 1
75         break
76
77 if start_index == -1:
78
79     print("\nFile does not contain required modal records.")
80     return
81
82 # =====
83 # INITIALIZE VARIABLES
84 # =====
85
86 previous_line = None
87 transition_line = None
88
89 prev_mode = None
90 prev_freq = None
91
92 trans_mode = None
93 trans_freq = None
94
95 # =====
96 # FIND TRANSITION
97 # =====
98
99 for line in lines[start_index:]:
100
101     parts = line.split()
102
103     if len(parts) < 3:
104         continue
105
106     try:
107
108         # =====
109         # ORIGINAL USER LOGIC (UNCHANGED)
110         # =====
111
112         mode = int(parts[1])
113         frequency = float(parts[2])
114
115         # =====
116         # RIGID BODY MODE
117         # =====
118
119         if abs(frequency) < 1e-2:
120
121             previous_line = line.strip()
122
123             prev_mode = mode
124             prev_freq = frequency
125
126             continue
127
128
129

```

```

129
130     # =====
131     # FIRST STRUCTURAL MODE
132     # =====
133
134     else:
135
136         transition_line = line.strip()
137
138         trans_mode = mode
139         trans_freq = frequency
140
141         break
142
143     except Exception:
144
145         continue
146
147     # =====
148     # RESULT FILE GENERATION
149     # =====
150
151     result_filename = filename.replace(".out", "_result.html")
152
153     result_path = os.path.join(post_path, result_filename)
154
155     now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
156
157     # =====
158     # HTML + CSS CONTENT
159     # =====
160

```

```

163 <!DOCTYPE html>
164
165 <html>
166
167 <head>
168
169 <meta charset="UTF-8">
170
171 <title>Eka Mobility Modal Analysis Report</title>
172
173 <style>
174
175 @import url('https://fonts.googleapis.com/css2?family=Montserrat:wght@400;500;600;700&display=swap');
176
177 *{
178     margin:0;
179     padding:0;
180     box-sizing:border-box;
181 }
182
183 body{
184     font-family:'Montserrat',sans-serif;
185     background:#edf3f8;
186     padding:40px;
187     color:#1a1a1a;
188 }
189
190 .container{
191     max-width:1400px;
192     margin:auto;
193     background:white;
194     border-radius:18px;
195     overflow:hidden;
196     box-shadow:0 10px 30px rgba(0,0,0,0.12);
197 }
198
199 /* ===== */
200 /* HEADER */
201 /* ===== */
202
203 .header{
204     background:linear-gradient(135deg,#005fa3,#0074bc);
205     color:white;
206     padding:30px 40px;
207
208     display:flex;
209     justify-content:space-between;
210     align-items:center;
211     flex-wrap:wrap;
212 }
213
214 .header-left{
215     display:flex;
216     align-items:center;
217     gap:25px;
218 }
219
220 .logo1{
221     width:90px;
222     background:white;
223     padding:10px;
224     border-radius:12px;
225 }
226
227 .logo2{
228     width:180px;
229 }
230
231 .title-group h1{
232     font-size:42px;
233     font-weight:700;
234 }
235
236 .title-group h2{
237     font-size:18px;
238     font-weight:400;
239     margin-top:6px;
240 }

```

```

242  /* ===== */
243  /* BLOCKS */
244  /* ===== */
245
246  .block{{
247      padding:35px 40px;
248      border-bottom:1px solid #dde7f0;
249  }}
250
251  .section-title{{
252      font-size:28px;
253      color:#005fa3;
254      margin-bottom:30px;
255      font-weight:700;
256      border-left:6px solid #0074bc;
257      padding-left:15px;
258  }}
259
260  /* ===== */
261  /* METRICS */
262  /* ===== */
263
264  .metrics{{
265      display:grid;
266      grid-template-columns:repeat(auto-fit,minmax(260px,1fr));
267      gap:22px;
268  }}
269
270  .card{{
271      background:#f8fbff;
272      border:1px solid #dbe8f3;
273      border-radius:16px;
274      padding:25px;
275  }}
276
277  .card-title{{
278      color:#6b7280;
279      font-size:14px;
280      margin-bottom:12px;
281      text-transform:uppercase;
282      letter-spacing:1px;
283  }}
284
285  .card-value{{
286      color:#0074bc;
287      font-size:22px;
288      font-weight:700;
289      word-break:break-word;
290  }}
291

```

```

325
326 /* ===== */
327 /* INTERPRETATION BLOCK */
328 /* ===== */
329
330 .interpretation-block{
331     margin-top:35px;
332 }
333
334 .interpretation-title{
335     font-size:28px;
336     color:#005fa3;
337     margin-bottom:25px;
338     font-weight:700;
339     border-left:6px solid #0074bc;
340     padding-left:15px;
341 }
342
343 .ok{
344     margin-top:25px;
345     background:#e8f7ec;
346     color:#1f7a38;
347     padding:16px;
348     border-radius:10px;
349     border-left:6px solid #28a745;
350     font-weight:600;
351 }
352
353 .warning{
354     margin-top:25px;
355     background:#fff4df;
356     color:#b7791f;
357     padding:16px;
358     border-radius:10px;
359     border-left:6px solid #ff9800;
360     font-weight:600;
361 }
362
363 .error{
364     margin-top:25px;
365     background:#ffe5e5;
366     color:#c62828;
367     padding:16px;
368     border-radius:10px;
369     border-left:6px solid #e53935;
370     font-weight:600;
371 }
372

```

```

373  /* ===== */
374  /* HASH */
375  /* ===== */
376
377  .hash-box{
378      margin-top:25px;
379      padding:20px;
380      background:#0b456b;
381      border-radius:12px;
382      color:white;
383      font-family:monospace;
384      word-break:break-all;
385      line-height:1.8;
386  }
387
388  .footer{
389      margin-top:30px;
390      text-align:center;
391      color:#d0d0d0;
392      font-size:13px;
393  }
394
395  @media(max-width:900px){
396
397      .summary-grid{
398          grid-template-columns:1fr;
399      }
400
401      .header{
402          flex-direction:column;
403          align-items:flex-start;
404          gap:20px;
405      }
406
407      .title-group h1{
408          font-size:32px;
409      }
410  }
411
412 </style>
413
414 </head>
415
416 <body>
417
418 <div class="container">
419
420     <!-- HEADER -->
421
422     <div class="header">
423
424         <div class="header-left">
425
426             
427
428             <div class="title-group">
429
430                 <h1>Modal Analysis Report</h1>
431
432                 <h2>R&D - CAE Department </h2>
433
434             </div>
435
436         </div>
437
438         
439
440     </div>
441
442     <!-- KEY METRICS -->
443
444     <div class="block">
445
446         <div class="section-title">
447             Key Metrics
448         </div>
449
450         <div class="metrics">
451
452             <div class="card">

```

```

453     <div class="card-title">File Name</div>
454     <div class="card-value">{filename}</div>
455 </div>
456
457 <div class="card">
458     <div class="card-title">Processed On</div>
459     <div class="card-value">{now}</div>
460 </div>
461
462 <div class="card">
463     <div class="card-title">Last Rigid Mode</div>
464     <div class="card-value">{prev_mode}</div>
465 </div>
466
467 <div class="card">
468     <div class="card-title">First Structural Mode</div>
469     <div class="card-value">{trans_mode}</div>
470 </div>
471
472 </div>
473
474 </div>
475
476 <!-- TRANSITION SUMMARY -->
477
478 <div class="block">
479
480     <div class="section-title">
481         Transition Summary
482     </div>
483
484     <div class="summary-grid">
485
486         <div class="summary-box">
487
488             <h3>Rigid Body Information</h3>
489
490             <div class="summary-item">
491                 <span>Last Rigid Mode :</span> {prev_mode}
492             </div>
493
494             <div class="summary-item">
495                 <span>Rigid Frequency :</span> {prev_freq:.6f} Hz
496             </div>
497
498         </div>
499
500         <div class="summary-box">
501
502             <h3>Structural Information</h3>
503
504             <div class="summary-item">
505                 <span>First Structural Mode :</span> {trans_mode}
506             </div>
507
508             <div class="summary-item">
509                 <span>Structural Frequency :</span> {trans_freq:.6f} Hz
510             </div>
511
512         </div>
513
514     </div>
515
516 <!-- INTERPRETATION BLOCK -->
517
518 <div class="interpretation-block">
519
520     <div class="interpretation-title">
521         Interpretation
522     </div>
523

```

```

524
525 # =====
526 # INTERPRETATION
527 # =====
528
529 if prev_mode == 6:
530
531     html_content += """
532
533     <div class="ok">
534
535         OK : System is properly constrained
536         with 6 rigid body modes.
537
538     </div>
539 """
540
541 elif prev_mode and prev_mode < 6:
542
543     html_content += """
544
545     <div class="warning">
546
547         WARNING : Under-constrained system detected.
548
549     </div>
550 """
551
552 elif prev_mode and prev_mode > 6:
553
554     html_content += """
555
556     <div class="warning">
557
558         WARNING : Possible over-constraint or
559         modeling issue detected.
560
561     </div>
562 """
563
564 else:
565
566     html_content += """
567
568     <div class="error">
569
570         ERROR : Unable to determine system interpretation.
571
572     </div>
573 """
574
575 # =====
576 # PRINT TRANSITION MODE
577 # =====
578
579 if trans_mode is not None:
580
581     html_content += f"""
582
583     <div class="summary-item"
584     style="
585         margin-top:22px;
586         font-size:18px;
587         font-weight:600;
588         color:#005fa3;
589     ">
590
591         Transition occurs at Mode : {trans_mode}
592
593     </div>
594 """
595

```



```

595
596     # =====
597     # SHA256 HASH
598     # =====
599
600     report_hash = hashlib.sha256(
601         | html_content.encode()
602     ).hexdigest()
603
604     html_content += f"""
605
606     </div>
607
608 </div>
609
610 <!-- HASH SECTION -->
611
612 <div class="block">
613
614     <div class="section-title">
615         | Integrity Verification
616     </div>
617
618     <div class="hash-box">
619
620         <b>SHA256 Hash:</b><br><br>
621
622         {report_hash}
623
624     </div>
625
626     <div class="footer">
627
628         | © Copyright Confidential eka Mobility India Pvt.Ltd
629
630     </div>
631
632 </div>
633
634 </div>
635
636 </body>
637
638 </html>
639 """
640
641     # =====
642     # WRITE HTML FILE
643     # =====
644
645     with open(result_path, "w", encoding="utf-8") as result_file:
646         | result_file.write(html_content)
647
648
649     # =====
650     # MAKE HTML FILE READ ONLY
651     # =====
652
653     os.chmod(result_path, 0o444)
654
655     # =====
656     # COPY LOGOS
657     # =====
658
659     current_dir = os.path.dirname(os.path.abspath(__file__))
660
661     shutil.copy(
662         | os.path.join(current_dir, "ekalogo1.jpeg"),
663         | post_path
664     )
665
666     shutil.copy(
667         | os.path.join(current_dir, "eka_logo_final.jpg"),
668         | post_path
669     )
670

```

```

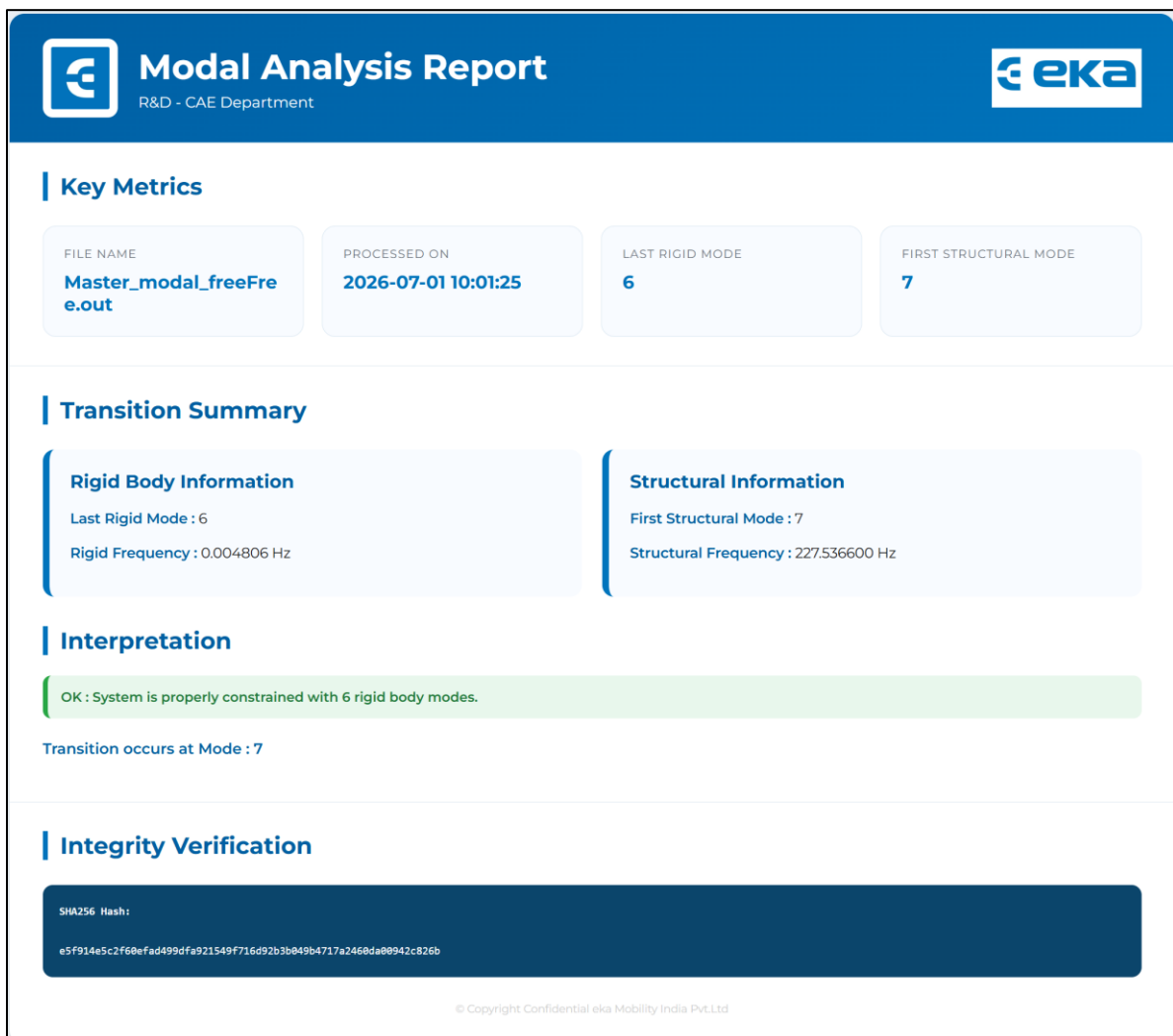
670
671     # =====
672     # SAVE HASH FILE
673     # =====
674
675     hash_file_path = result_path + ".sha256"
676
677     with open(hash_file_path, "w") as hash_file:
678
679         hash_file.write(report_hash)
680
681     # =====
682     # MAKE HASH FILE READ ONLY
683     # =====
684
685     os.chmod(hash_file_path, 0o444)
686
687     # =====
688     # FILE MOVE SECTION (ORIGINAL LOGIC)
689     # =====
690
691     destination = os.path.join(post_path, filename)
692
693     shutil.move(file_path, destination)
694
695     print(f"File moved to post-process folder:\n{destination}")
696
697     # =====
698     # PRINT STATUS
699     # =====
700
701     print(f"\nResult saved at:\n{result_path}")
702
703     print(f"\nSHA256 file saved at:\n{hash_file_path}")
704
705     except Exception as e:
706
707         print("\n=====")
708         print("ERROR OCCURRED")
709         print("=====")
710         print(e)
711         print("=====\n")
712
713     # =====
714     # RUN
715     # =====
716
717     check_and_move_file()

```

Results:

The developed automation tool successfully extracted the essential modal analysis information directly from the OptiStruct solver output file and generated a concise HTML summary report. The report displayed the total number of rigid body modes, the mode number corresponding to the first flexible natural frequency, and the value of the first natural frequency in a clear and organized format.

Compared to manual inspection of the solver output files, the automated approach significantly reduced the time required for result verification while eliminating the possibility of overlooking important modal information. The tool improved the efficiency of the post-processing workflow and enabled engineers to review simulation results more quickly and consistently.



Automation of LS-DYNA ELout Beam Force Extraction and Bolt Force Summary Generation

Introduction:

LS-DYNA is a widely used explicit finite element solver for nonlinear dynamic analyses such as crash, impact, and drop simulations. During analysis, the solver generates an ELout file containing element-wise force results at every output time step. For models in which bolts are represented using one-dimensional beam elements, the ELout file contains the axial and shear force components for each beam throughout the simulation. Since these output files are large and contain results for numerous time steps, manually extracting and evaluating the force history of each bolt is both time-consuming and prone to error.

Objective:

The objective of this project was to develop a Python-based automation tool capable of extracting beam force data from the LS-DYNA ELout file and generating a structured Microsoft Excel report. The tool extracts the axial force, shear force in the S-direction, and shear force in the T-direction for every beam at each time step. It also calculates the resultant shear force, identifies the maximum force values along with their corresponding time instances, and summarizes the results in an easily interpretable format. Additionally, since each physical bolt is represented by two consecutive beam elements, the tool combines their individual forces to determine the resultant bolt forces.

Methodology:

The Python script reads the ELout file and scans the complete solver output to identify every beam element and its associated time history. For each beam, the axial force, shear-S force, and shear-T force are extracted at every output time step. The script then calculates the resultant shear force using the two shear components and stores the complete force history for each beam.

After extracting the beam data, the script groups every two consecutive beam elements into a single bolt based on the beam numbering convention used in the finite element model. Using vector calculations, it determines the resultant axial force and resultant shear force acting on each bolt throughout the simulation. The maximum axial and shear forces, together with the corresponding time at which they occur, are automatically identified.

Finally, the extracted data is organized into a Microsoft Excel workbook. The first worksheet contains the complete time-history data for every beam and resultant bolt, including the maximum force values and their occurrence times. A second worksheet serves as a dashboard summarizing the maximum axial and shear forces for every bolt. The dashboard also includes automatically generated line graphs illustrating the variation of axial and shear forces with time, enabling rapid visualization and comparison of bolt loading during the simulation.

Role of Python:

Python was used to automate the complete post-processing workflow of the LS-DYNA simulation results. It reads the ELout solver output, extracts the required beam force information, performs the mathematical calculations required to obtain resultant bolt forces, identifies the critical load values, and automatically generates a formatted Excel workbook containing summary tables and graphical plots. This automation eliminates repetitive manual calculations, significantly reduces post-processing time, improves the accuracy and consistency of the results, and provides engineers with a standardized report for evaluating bolt forces during crash simulations.

Dependencies:

1. Python 3.14.5
2. Openpyxl Library
3. Math Library
4. Workbook Library

Source Code:

```
1 import os
2 import re
3 import math
4 from openpyxl import Workbook
5 from openpyxl.styles import Border, Side, Font, Alignment
6 from openpyxl.utils import get_column_letter
7 from openpyxl.chart import LineChart, Reference
8
9 # =====
10 # EXTRACT TIME VS AXIAL FORCE
11 # =====
12
13 def extract_all_beam_forces(elout_file):
14
15     beam_data = {}
16
17     with open(elout_file, "r", errors="ignore") as file:
18
19         lines = file.readlines()
20
21         current_time = None
22
23         total_lines = len(lines)
24
25         for i in range(total_lines):
26
27             line = lines[i]
28
29             # =====
30             # 1. EXTRACT TIME
31             # =====
32             time_match = re.search(
33                 r'\(s*at time\s*([+-]?\d+\.\d+E[+-]?\d+)\s*\)',
34                 line,
35                 re.IGNORECASE
36             )
37
38             if time_match:
39
40                 current_time = float(
41                     time_match.group(1)
42                 )
43
44             # =====
45             # 2. DETECT BEAM/TRUSS BLOCK
46             # =====
47             beam_match = re.search(
48                 r'beam/truss\s*\s*=\s*(\d+)',
49                 line,
50                 re.IGNORECASE
51             )
52
53             if beam_match:
54
55                 beam_id = int(
56                     beam_match.group(1)
57                 )
58
59                 # initialize beam if not exists
60                 if beam_id not in beam_data:
61
62                     beam_data[beam_id] = []
63
64                 # =====
65                 # 3. SEARCH RESULTANT BLOCK (LOOK AHEAD)
66                 # =====
67
68                 for j in range(i + 1, total_lines):
69
70                     next_line = lines[j]
71
72                     # stop if next beam starts
73                     if "beam/truss" in next_line.lower():
74                         break
75
76                     # find resultants block
77                     if "resultants" in next_line.lower():
78
```

```

78
79
80 # =====
81 # 4. READ AXIAL FORCE VALUE
82 # =====
83
84 for k in range(j + 1, total_lines):
85     value_line = lines[k].strip()
86
87     if not value_line:
88         continue
89
90     values = value_line.split()
91
92     try:
93
94         axial_force = float(values[0])
95
96         shear_s = float(values[1])
97
98         shear_t = float(values[2])
99
100         shear_resultant = math.sqrt(
101             shear_s**2 +
102             shear_t**2
103         )
104
105         if current_time is not None:
106
107             beam_data[beam_id].append(
108                 (
109                     current_time,
110                     axial_force,
111                     shear_s,
112                     shear_t,
113                     shear_resultant
114                 )
115             )
116
117             break
118
119         except (ValueError, IndexError):
120
121             continue
122
123     break
124
125     return beam_data
126
127 # =====
128 # CREATE EXCEL
129 # =====
130
131 def create_excel(
132     all_beams,
133     resultant_beams,
134     output_excel
135 ):
136
137     wb = Workbook()
138
139     ws = wb.active
140     ws.title = "Beam Forces"
141
142     dashboard = wb.create_sheet("Dashboard")
143

```

```

144 # =====
145 # DASHBOARD TABLE
146 # =====
147
148 dashboard.cell(row=1, column=1, value="Bolt")
149 dashboard.cell(row=1, column=2, value="Max Axial")
150 dashboard.cell(row=1, column=3, value="Time @ Max Axial")
151 dashboard.cell(row=1, column=4, value="Max Shear")
152 dashboard.cell(row=1, column=5, value="Time @ Max Shear")
153
154 for col in range(1, 6):
155
156     dashboard.cell(
157         row=1,
158         column=col
159     ).font = Font(
160         name="Cambria",
161         size=12,
162         bold=True
163     )
164
165 dash_row = 2
166
167 for beam_id, data in all_beams.items():
168
169     if not str(beam_id).startswith("Resultant"):
170         continue
171
172     results = calculate_results(data)
173
174     dashboard.cell(
175         row=dash_row,
176         column=1,
177         value=str(beam_id)
178     )
179
180     dashboard.cell(
181         row=dash_row,
182         column=2,
183         value=results["axial_max"][1]
184     )
185
186     dashboard.cell(
187         row=dash_row,
188         column=3,
189         value=results["axial_max"][0]
190     )
191
192     dashboard.cell(
193         row=dash_row,
194         column=4,
195         value=results["shear_max"][2]
196     )
197
198     dashboard.cell(
199         row=dash_row,
200         column=5,
201         value=results["shear_max"][0]
202     )
203
204     dash_row += 1
205
206 start_col = 1
207
208 for beam_id, data in all_beams.items():
209
210     if not data:
211         continue
212
213     results = calculate_results(data)
214
215     row = 1
216     col = start_col

```



```

217
218 # =====
219 # BEAM HEADING
220 # =====
221
222 ws.merge_cells(
223     start_row=row,
224     start_column=col,
225     end_row=row,
226     end_column=col + 2
227 )
228
229 heading_cell = ws.cell(
230     row=row,
231     column=col,
232     value=f"Beam {beam_id}"
233 )
234
235 heading_cell.font = Font(
236     name="Cambria",
237     size=12,
238     bold=True
239 )
240
241 heading_cell.alignment = Alignment(
242     horizontal="center",
243     vertical="center",
244     wrap_text=True
245 )
246
247 row += 1
248
249 # =====
250 # TABLE HEADER
251 # =====
252
253 time_header = ws.cell(
254     row=row,
255     column=col,
256     value="Time (s)"
257 )
258
259 time_header.font = Font(
260     name="Cambria",
261     bold=True
262 )
263
264 time_header.alignment = Alignment(
265     horizontal="center"
266 )
267
268
269 force_header = ws.cell(
270     row=row,
271     column=col + 1,
272     value="Axial Force"
273 )
274
275 force_header.font = Font(
276     name="Cambria",
277     bold=True
278 )
279
280 force_header.alignment = Alignment(
281     horizontal="center"
282 )
283
284
285 shear_header = ws.cell(
286     row=row,
287     column=col + 2,
288     value="Shear Resultant"
289 )
290
291 shear_header.font = Font(
292     name="Cambria",
293     bold=True
294 )
295
296 shear_header.alignment = Alignment(

```

```

297         horizontal="center"
298     )
299
300     row += 1
301
302     # =====
303     # FORCE DATA
304     # =====
305
306     for row_data in data:
307
308         time_value = row_data[0]
309         axial_force = row_data[1]
310         shear_force = row_data[-1]
311
312         ws.cell(
313             row=row,
314             column=col,
315             value=time_value
316         )
317
318         ws.cell(
319             row=row,
320             column=col + 1,
321             value=axial_force
322         )
323
324         ws.cell(
325             row=row,
326             column=col + 2,
327             value=shear_force
328         )
329
330         row += 1
331
332     # =====
333     # MAXIMUM FORCE
334     # =====
335
336     ws.cell(
337         row=row,
338         column=col,
339         value="Maximum Axial Force"
340     )
341
342     ws.cell(
343         row=row,
344         column=col + 1,
345         value=results["axial_max"][1]
346     )
347
348     row += 1
349
350     ws.cell(
351         row=row,
352         column=col,
353         value="Time @ Max Axial"
354     )
355
356     ws.cell(
357         row=row,
358         column=col + 1,
359         value=results["axial_max"][0]
360     )
361
362     row += 1
363
364     ws.cell(
365         row=row,
366         column=col,
367         value="Maximum Shear Force"
368     )
369
370     ws.cell(
371         row=row,
372         column=col + 1,
373         value=results["shear_max"][2]
374     )
375

```

```

376     row += 1
377
378     ws.cell(
379         row=row, (variable) col: int
380         column=col,
381         value="Time @ Max Shear"
382     )
383
384     ws.cell(
385         row=row,
386         column=col + 1,
387         value=results["shear_max"][0]
388     )
389
390     # Leave 2 blank columns between tables
391     start_col += 5
392
393     # =====
394     # BORDERS
395     # =====
396
397     thin_border = Border(
398         left=Side(style="thin"),
399         right=Side(style="thin"),
400         top=Side(style="thin"),
401         bottom=Side(style="thin")
402     )
403
404     for row_cells in ws.iter_rows(
405         min_row=1,
406         max_row=ws.max_row,
407         min_col=1,
408         max_col=ws.max_column):
409
410         for cell in row_cells:
411
412             if cell.value is not None:
413
414                 cell.border = thin_border
415
416     # =====
417     # COLUMN WIDTHS
418     # =====
419
420     for col_num in range(1, ws.max_column + 1):
421
422         ws.column_dimensions[
423             get_column_letter(col_num)
424         ].width = 25
425
426     for row_cells in ws.iter_rows():
427
428         for cell in row_cells:
429
430             if cell.value is not None:
431
432                 cell.font = Font(
433                     name="Cambria",
434                     size=11,
435                     bold=cell.font.bold
436                 )
437
438                 cell.alignment = Alignment(
439                     horizontal="center",
440                     vertical="center"
441                 )

```

```

442 # =====
443 # DASHBOARD CHARTS
444 # =====
445
446 graph_start_row = 20
447 chart_position_row = 2
448
449 for bolt_name, bolt_data in resultant_beams.items():
450
451     if not str(bolt_name).startswith("Resultant"):
452         continue
453
454     current_row = graph_start_row
455
456     dashboard.cell(current_row, 1, "Time")
457     dashboard.cell(current_row, 2, "Axial")
458     dashboard.cell(current_row, 3, "Shear")
459
460     current_row += 1
461
462     for row_data in bolt_data:
463
464         dashboard.cell(current_row, 1, row_data[0])
465         dashboard.cell(current_row, 2, row_data[1])
466         dashboard.cell(current_row, 3, row_data[2])
467
468         current_row += 1
469
470     # -----
471     # Axial Chart
472     # -----
473
474     axial_chart = LineChart()
475     axial_chart.title = f"{bolt_name} Axial"
476
477     data = Reference(
478         dashboard,
479         min_col=2,
480         min_row=graph_start_row,
481         max_row=current_row - 1
482     )
483
484     categories = Reference(
485         dashboard,
486         min_col=1,
487         min_row=graph_start_row + 1,
488         max_row=current_row - 1
489     )
490
491     axial_chart.add_data(data, titles_from_data=True)
492     axial_chart.set_categories(categories)
493
494     dashboard.add_chart(
495         axial_chart,
496         f"G{chart_position_row}"
497     )
498
499     # -----
500     # Shear Chart
501     # -----
502
503     shear_chart = LineChart()
504     shear_chart.title = f"{bolt_name} Shear"
505
506     data = Reference(
507         dashboard,
508         min_col=3,
509         min_row=graph_start_row,
510         max_row=current_row - 1
511     )
512
513     shear_chart.add_data(data, titles_from_data=True)
514     shear_chart.set_categories(categories)
515
516     dashboard.add_chart(
517         shear_chart,
518         f"O{chart_position_row}"
519     )
520

```

```

521         graph_start_row = current_row + 5
522         chart_position_row += 15
523
524     # =====
525     # HIDE GRAPH DATA
526     # =====
527
528     # for r in range(20, current_row + 1):
529     #     dashboard.row_dimensions[r].hidden = True
530
531     # =====
532     # DASHBOARD FORMATTING
533     # =====
534
535     for col_num in range(1, 6):
536
537         dashboard.column_dimensions[
538             get_column_letter(col_num)
539         ].width = 25
540
541     for row_cells in dashboard.iter_rows(
542         min_row=1,
543         max_row=dash_row - 1,
544         min_col=1,
545         max_col=5):
546
547         for cell in row_cells:
548
549             if cell.value is not None:
550
551                 cell.border = thin_border
552
553                 cell.font = Font(
554                     name="Cambria",
555                     size=11,
556                     bold=(cell.row == 1)
557                 )
558
559                 cell.alignment = Alignment(
560                     horizontal="center",
561                     vertical="center"
562                 )
563
564     wb.save(output_excel)
565
566     # =====
567     # CALCULATE RESULTS
568     # =====
569
570
571     def calculate_results(data):
572
573         axial_max = max(
574             data,
575             key=lambda x: abs(x[1])
576         )
577
578         shear_max = max(
579             data,
580             key=lambda x: abs(x[-1])
581         )
582
583         return {
584             "axial_max": axial_max,
585             "shear_max": shear_max
586         }
587
588     # =====
589     # RESULTANT CALCULATION
590     # =====
591
592     def calculate_resultant_beams(all_beams):
593
594         resultant_beams = {}
595
596         beam_ids = sorted(all_beams.keys())
597
598         for i in range(0, len(beam_ids) - 1, 2):
599

```

```

600     beam1 = beam_ids[i]
601     beam2 = beam_ids[i + 1]
602
603     data1 = all_beams[beam1]
604     data2 = all_beams[beam2]
605
606     resultant_data = []
607
608     print(data1[0])
609
610     for (
611         time1,
612         axial1,
613         shear_s1,
614         shear_t1,
615         shear_res1
616     ), (
617         time2,
618         axial2,
619         shear_s2,
620         shear_t2,
621         shear_res2
622     ) in zip(
623         data1,
624         data2):
625
626         bolt_axial = math.sqrt(
627             axial1**2 +
628             axial2**2
629         )
630
631         bolt_shear = math.sqrt(
632             shear_res1**2 +
633             shear_res2**2
634         )
635
636         resultant_data.append(
637             (
638                 time1,
639                 bolt_axial,
640                 bolt_shear
641             )
642         )
643
644     resultant_beams[
645         f"Resultant {beam1}-{beam2}"
646     ] = resultant_data
647
648     return resultant_beams
649
650 # =====
651 # MAIN
652 # =====
653
654 def main():
655
656     print()
657     print("=====")
658     print("LS-DYNA ELOUT Beam Force Extractor")
659     print("=====")
660     print()
661
662     elout_file = input(
663         "Enter ELOUT file path : "
664     ).strip()
665
666     print()
667     print("Reading entire ELOUT file and extracting all beams...")
668     print()
669
670     # =====
671     # CREATE OUTPUT FOLDER
672     # =====
673
674     output_folder = r"Z:\09.1_User CAE_Data\22_Yash_Intern\From_Lalit\01_Beam_forces\Output"
675
676     os.makedirs(
677         output_folder,
678         exist_ok=True

```

```

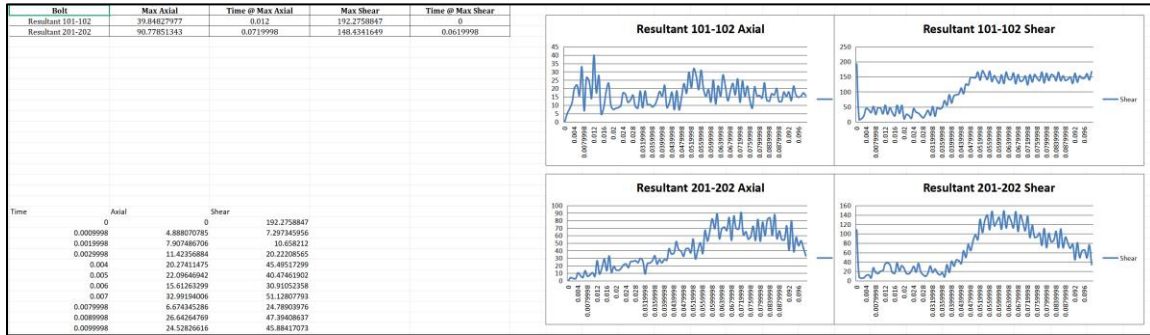
679     )
680
681     # Step 1: extract everything in one scan
682     all_beams = extract_all_beam_forces(elout_file)
683
684     resultant_beams = calculate_resultant_beams(
685         all_beams
686     )
687
688     if not all_beams:
689         print("No beam data found in file")
690         return
691
692     base_name = os.path.splitext(
693         os.path.basename(elout_file)
694     )[0]
695
696     excel_file = os.path.join(
697         output_folder,
698         f"{base_name}_Beam_Summary.xlsx"
699     )
700
701     all_beams.update(resultant_beams)
702
703     create_excel(
704         all_beams,
705         resultant_beams,
706         excel_file
707     )
708
709     print()
710     print("Excel summary created successfully")
711     print(os.path.abspath(excel_file))
712
713 if __name__ == "__main__":
714     main()

```

Results:

The developed automation tool successfully converted the extensive ELout solver output into a concise and organized Excel report. The script automatically extracted the force history of every beam, calculated the resultant forces for each bolt, identified the maximum axial and shear loads together with their corresponding time instances, and generated graphical representations of the force variation throughout the simulation. Compared to manual post processing, the automated workflow substantially reduced the time required for result extraction and organization while eliminating manual calculation errors. This enabled faster assessment of bolt performance and improved the overall efficiency of the LS-DYNA post-processing process.

Results:



Beam 101		
Time (s)	Axial Force	Shear Resultant
0	0	187.2031791
0.0009998	3.3	3.614745496
0.0019998	7.761	5.153062196
0.0029998	11.37	2.405587662
0.004	19.74	7.51122287
0.005	22.06	4.565000438
0.006	15.49	14.48204972
0.007	32.97	16.4027755
0.0079998	5.258	19.74965407
0.0089998	26.58	11.55192183
0.0099998	24.44	9.640423227
0.011	13.69	24.08773431
0.012	39.82	13.67427746
0.013	13.44	25.42038806
0.014	27.22	12.04976763
0.015	4.518	4.093856983
0.016	7.715	12.62899018
0.017	17.12	23.65176038
0.018	22.23	28.21076306
0.019	8.211	18.69345204
0.02	5.483	7.544040109
0.021	5.579	12.426094
0.022	8.537	12.27917082
0.023	9.142	10.44526955
0.024	16.84	33.19457335
0.025	14.29	25.86253275
0.026	11.23	21.49831828
0.027	12.76	15.63535097
0.028	12.11	8.99983311
0.029	7.895	7.405588228
0.03	8.048	16.3929778
0.031	12.88	17.29117983
0.0319998	8.542	24.34475393
0.0329998	17.85	13.72624643
0.0339998	10.22	13.70522634
0.0349998	9.165	8.439262112
0.0359998	8.549	15.00711568
0.0369998	9.837	9.539634427
0.0379998	13.05	17.25345707
0.0389998	18.17	12.39370824
0.0399998	15.18	7.81473736
0.0409998	19.56	4.202810964
0.0419998	8.504	16.00650933
0.0429998	11.41	8.558409023
0.0439998	17.75	8.811745868
0.0449998	7.303	12.41925254
0.0459998	18.41	8.938687698
0.0469998	6.629	8.392604006
0.0479998	15.01	2.644000025
0.0489998	22.29	9.072763979
0.0499998	16.57	4.16056078
0.0509998	29.47	21.73675689
0.0519998	20.44	17.07714847
0.0529998	31.74	16.02443447
0.0539998	27.36	3.683000416
0.0549998	19.04	3.365352879

Nonlinear Buckling Analysis of the Drag Link of the Steering System

Introduction:

The steering system is one of the most critical vehicle subsystems, responsible for transmitting the driver's steering input to the road wheels while ensuring accurate directional control and vehicle stability. In heavy commercial vehicles such as buses, the steering linkage consists of several interconnected components that transfer motion from the steering gearbox to the steering knuckles.

A **drag link** is a key component of the steering linkage that connects the steering gearbox or pitman arm to the steering mechanism. Its primary function is to transmit steering forces and motion while maintaining precise wheel alignment during vehicle operation. Since the drag link is subjected to significant compressive and tensile loads arising from steering inputs and road conditions, its structural integrity is essential for ensuring safe and reliable steering performance.

To evaluate its stability under compressive loading, a nonlinear buckling analysis was performed using finite element analysis. Two separate simulation iterations were carried out using different mesh discretization techniques—a hexahedral mesh and a tetrahedral mesh—to study the influence of mesh type on the predicted buckling response. The analyses were compared with respect to the structural response and overall solution characteristics. Due to company confidentiality policies, the specific component geometry, vehicle model, loading conditions, and numerical results have not been disclosed in this report.

Buckling analysis is a critical aspect of structural design used to evaluate the stability of components subjected to compressive loading. In practical engineering applications, geometric imperfections, material nonlinearities, and large deformations can significantly influence buckling behaviour. Therefore, nonlinear buckling analysis is often preferred over linear eigenvalue buckling analysis as it provides a more realistic prediction of structural stability under service loading conditions.

Objective:

The objective of this study was to perform a nonlinear buckling analysis on a structural component using finite element analysis and to compare the influence of different mesh types on the predicted structural response. Due to company confidentiality policies, the component name, geometry, loading conditions, and numerical results have not been disclosed in this report.

Methodology:

The finite element model was prepared using **Altair HyperWorks**, where the geometry was cleaned, meshed, and assigned the required material properties, boundary conditions, and loading conditions. Two separate mesh configurations were developed for the study. The first iteration employed a predominantly hexahedral mesh, while the second iteration utilized a tetrahedral mesh. Both models were created using comparable mesh quality criteria to facilitate a meaningful comparison.

The nonlinear buckling analysis was carried out using the **OptiStruct** solver with geometric non-linearity considered in the solution. Both mesh configurations were analyzed under identical loading and boundary conditions to evaluate the effect of mesh type on the structural response. After the analysis, the simulation results were post-processed in **Altair HyperWorks**, where the displacement contours, stress distributions, and buckled deformation patterns were examined and compared for the two mesh configurations.

Methodology used for pre-processing

CAD Geometry



Geometry Cleanup



Mesh Generation in HyperWorks



Material Assignment



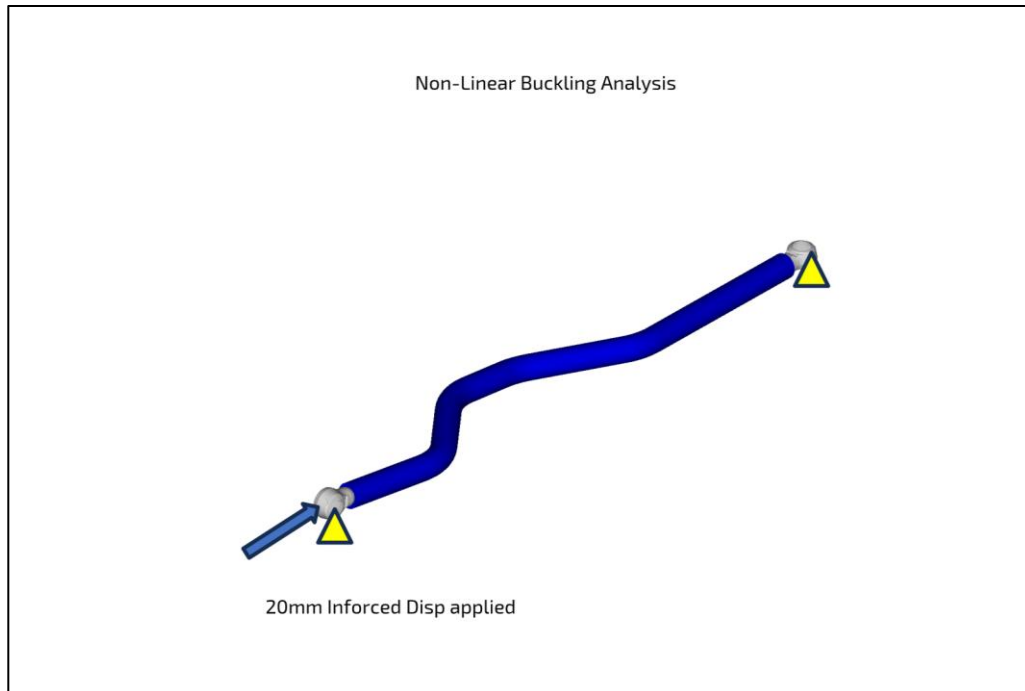
Boundary Conditions



OptiStruct Solver



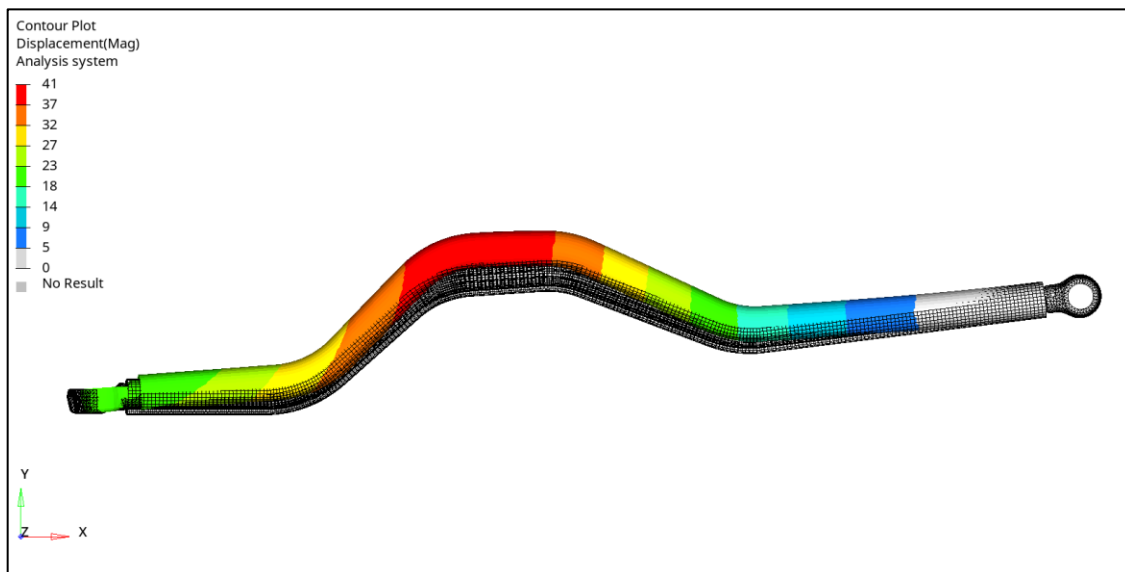
Post-processing in HyperWorks



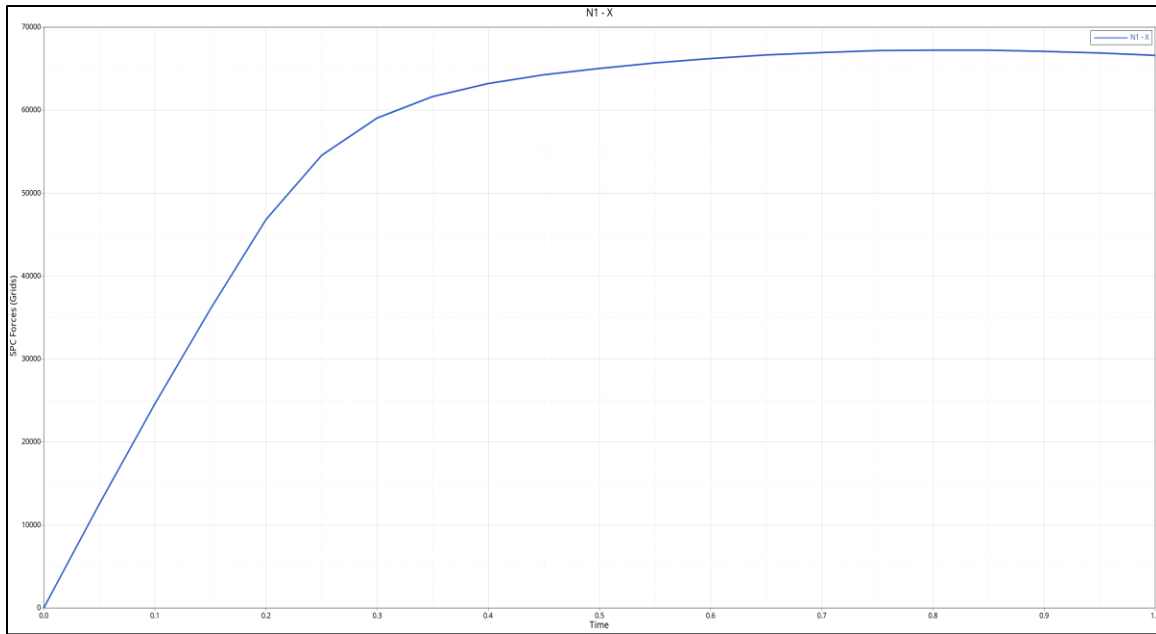
Loading Conditions (Triangles denote SPC)

Results and Discussion:

The nonlinear buckling analysis provided valuable insight the structural stability of the component, it highlighted the design feasibility and scope for improvements. It also provided insights on a possible failure conditions if it may arise.



Deformation



Graph of Time vs Single Point Constraint (SPC) Force

Loadcase: Non-Linear Buckling Analysis			
Sr No	Component	Buckling Factor	Critical Buckling Load (kN)
1	Draglink	3.2	67.249

Result Summary

Driver Ergonomics Evaluation of Bus Platforms

Introduction:

Driver ergonomics is a key aspect of commercial vehicle design, directly influencing driver comfort, safety, and operational efficiency. A well-designed driver's workplace minimizes fatigue, improves visibility, provides easy access to vehicle controls, and enables the driver to maintain a comfortable posture during extended driving periods. To ensure these objectives are achieved, bus manufacturers design the driver's compartment in accordance with established national and international ergonomic standards.

Objective:

The objective of this study was to evaluate the driver ergonomics of existing **7 m city bus** and **12 m low-floor city bus** platforms by carrying out physical measurements on prototype vehicles and verifying their compliance with applicable automotive regulations and ergonomic standards.

Methodology:

The study involved conducting physical measurements on the prototype vehicles to assess various ergonomic parameters of the driver's workplace. Measurements related to the driver's seating position, steering wheel location, pedal arrangement, control accessibility, instrument panel layout, visibility, and workspace dimensions were recorded using suitable measuring instruments. The collected data was documented systematically for further evaluation.

The measured parameters were verified against the requirements specified in **AIS-052**, **UNECE Regulation No. 107**, **UNECE Regulation No. 35**, **ISO 16121 (Parts 1-3)**, **ISO 4040**, and **ISO 6549**. Each measurement was compared with the prescribed limits to assess compliance with the relevant ergonomic and regulatory requirements. Any deviations observed during the evaluation were documented and communicated for engineering review and design consideration.

SAE Standards for Ergonomics Driver Cabin Design	
Document Number	Title
J1521_202412	Truck Driver Shin-Knee Position for Clutch and Accelerator
J287_202211	Driver Hand Control Reach
J3099-202504	Driver Seat Accommodation and Steering Wheel Location Models
J1522_202412	Truck Driver Stomach Position
J1834_202210	Seat Belt comfort, fit and Convenience - Truck & Bus
J1517_202412	Driver Selected Seat Position for Class B Vehicles (HCV Truck and Buses) - Seat Track Length & SgRP

Following Measurements were taken into consideration for evaluation:

1. According to AIS-052 dimensions A, B, C, D, E & F and general dimensions specified as per ISO 16121-1 regarding driver work area.
2. Visibility zone measurements according to AIS-052 and ISO 16121-2.
3. Foot control Arrangement measurements according to UNECE R-35.
4. Dashboard measurements from H (Hip) point and shoulder points according to ISO 16121-3.
5. Steering wheel controls dimensions according to ISO 4040.

Results:

The ergonomics assessment provided a comprehensive evaluation of the driver's workplace for both bus platforms with respect to applicable national and international standards. The study assisted in verifying compliance with ergonomic requirements and identifying areas requiring design improvement wherever necessary. The assessment contributed to enhancing driver comfort, accessibility, visibility, and overall operational safety while supporting compliance with established automotive regulations. **Specific measurement values and detailed evaluation results have been omitted from this report due to company confidentiality requirements.**

Acknowledgement

I would like to express my sincere gratitude to **EKA Mobility** for providing me with the opportunity to undertake my internship and gain valuable exposure to the field of Computer-Aided Engineering (CAE). This internship has been an enriching learning experience that has strengthened my technical knowledge and provided me with practical insights into the automotive industry.

I extend my heartfelt thanks to the entire **CAE Department** for their constant support, guidance, and willingness to share their knowledge throughout my internship. Their encouragement and collaborative work environment made this experience both enjoyable and educational.

I would especially like to thank my **Manager Mr. Pratap Patil** for providing me with the opportunity to work on challenging engineering tasks and for offering valuable guidance and constructive feedback during the course of my internship.

I am also deeply grateful to my **Department Head- CAE, Mr. Kiran Karale** for their mentorship, technical insights, and encouragement, which played a significant role in enhancing my understanding of engineering analysis and professional practices.

Finally, I would like to thank everyone at EKA Mobility who contributed to my learning and made my internship a rewarding and memorable experience. The knowledge and experience gained during this period will undoubtedly serve as a strong foundation for my future professional career.